2023 International Conference on Computer, Control, Informatics and its Applications (IC3INA) | 979-8-3503-9487-0/23/$31.00 ©2023 IEEE | DOI: 10.1109/IC3INA60834.2023.10285736

Enhancing Machine Learning Models through Hyperparameter Optimization with Particle Swarm Optimization

Ariani Indrawati[1], Intan N. Wahyuni[2]
[1]*Research Center for Data and Information Science, National Research and Innovation Agency*
[2]*Research Center for Computing, National Research and Innovation Agency*
aria006@brin.go.id, inta008@brin.go.id

***Abstract*—Hyperparameter tuning plays a crucial role in optimizing the performance of machine learning algorithms. This study explores the effectiveness of Particle Swarm Optimization (PSO) in fine-tuning the hyperparameters of three popular machine learning algorithms, including Random Forest, K-Nearest Neighbors, and Support Vector Machine. The studies use two publicly available datasets: the Heart Failure Clinical Records Dataset and the Iris Dataset. For additional comparative analysis, we also compare with the common optimization technique using Random Search. The result reveals that PSO and Random Search have varying impacts on the performance of different algorithms and datasets. The findings provide valuable insights into the behavior of different optimization techniques and their effect on model performance, paving the way for better and more reliable machine learning models in various domains. Through effective hyperparameter optimization, machine learning models can achieve enhanced accuracy.**

***Index Terms*—Hyperparameter tuning, Particle Swarm Optimization, Machine Learning, Algorithms**

I. Introduction

Machine learning algorithms have become popular in solving complex problems across various domains [1]. These algorithms facilitate machine learning by allowing computers to acquire knowledge from data and subsequently generate predictions or make decisions. They have been used in various applications, including recommendation systems, image or speech recognition, natural language processing, and many more [2]. One of the issues in machine learning is optimizing the performance of these algorithms to generate high accuracy. The machine learning algorithm's performance is heavily influenced by its hyperparameters [3]. Hyperparameters are the configuration variables that are used to control the learning process. They affect various aspects of the learning process, such as the speed of convergence, the complexity of the model, and the trade-off between overfitting and underfitting. For example, the number of nearest neighbors is considered in the classification process. The model will overfit if $n_neighbor$ is too large and also costs a high computational cost. However, the model will be underfitting if $n_neighbor$ is too small.

Finding the optimal values for hyperparameters is essential for maximizing the performance of a machine-learning model. However, determining the optimal combination of those parameters is a challenging task. It systematically explores different combinations of the model's hyperparameter values and evaluates their impact on the algorithm's performance. Some hyperparameter optimization techniques can be used to search for the best combination, including model-free algorithms, gradient-based optimization, bayesian optimization, multi-fidelity algorithms, and metaheuristic algorithms [3].

In this study, we studied PSO, a metaheuristic algorithm, to optimize the machine learning model. PSO was proposed in 1995 by Kennedy and Eberhart, inspired by how fish and birds interact with one another to find food [4]. The basic concept of PSO is to maintain a population of particles that move effectively through a search space to find the best solution. In the context of hyperparameter tuning, PSO potentially improves the machine learning algorithm's performance by finding the optimal combination of hyperparameter values. Combining machine learning and PSO provides a robust framework for solving complex optimization problems.

The primary aim of this study is to explore the application of PSO for optimizing multiple machine-learning algorithms. Because of its simple concept, ease of implementation, and rapid convergence, PSO has been proven to have high optimization capabilities [5], [6]. We tried to optimize three algorithms, including Random Forest (RF) [7], Support Vector Machine (SVM) [8], and KNN [9]. To analyze the effectiveness of PSO in terms of solution quality, we do experiments on a few benchmark datasets from UCI, are Iris [10] and Heart Failure Clinical [11]. Furthermore, we will also compare the precision of enhanced machine-learning models by PSO with the common optimization technique, Random Search. Random search works by testing various combinations of parameters randomly from a predetermined range of values. This method is easy to implement, can overcome variations in data, and tends to be more efficient than other optimization methods [12]. By gaining insights into these optimization techniques, researchers and practitioners can leverage PSO to enhance the capabilities of their machine-learning models and tackle challenging real-world problems.

II. Related Works

A. Particle Swarm Optimization

Particle Swarm Optimization (PSO) is a metaheuristic algorithm frequently used to solve optimization issues [4]. This method is a study of the behavior of flocks of birds or schools

Authorized licensed use limited to: BMS College of Engineering. Downloaded on October 24,2026 at 07:14:19 UTC from IEEE Xplore. Restrictions apply.

of fish in survival, which shows individual and social behavior [13]. In PSO, the population is called a swarm, and the individuals are particles. Every particle moves in a particular space based on its position, velocity, and the history of the best place it has ever been. The particle then communicates its best potion information to other particles and adjusts each particle's position and velocity by the information received [14].

The PSO algorithm from [15] starts by randomly initializing each particle i in the swarm (S), where each particle has some values to solve the specific problem. Then, the inertia weight w, the constant coefficients c_1 and c_2, and the random numbers r_1 and r_2 are also determined. Construct each particle's position $X_k(i)$ and velocity $V_k(i)$ randomly for k iteration. After that, calculate each particle's fitness (F). Then compare each particle's output for each iteration with the previous iteration, and pick the particles with the best value between them to determine the local best value ($pBest$). The particle with the best fitness value is selected from all the particles and marked as $gbest$. Next, the particles will move based on their current position, the $gBest$, and their $pBest$. Position and velocity with $pbest$ and $gbest$ are updated using the equation below:

$$V_k(i) = wV_k(i-1)+c_1r_1(S_{pbestk}-S_k(i)+c_2r_2(S_{gbest}-S_k(i))$$

$$S_k(i) = S_k(i-1) + V_k(i)$$

$$i = i + 1$$

Repeat those steps until the maximum iteration is reached. The values in the last $gBest$ particles are considered the most optimal for the problem.

B. Hyper-parameters in Machine Learning Algorithms

1) **Random forest (RF)** is derived from a collection of decision tree algorithms, then combines these trees to obtain more accurate and stable predictions. The model can solve the solution of regression and classification problems. RF hyperparameters must tune for model accuracy. $n-estimator$ is the number of trees in the forest. If the number of trees increases, the time complexity of the model will increase, although it will not reduce the model's performance. max_depth is the longest path between the root and leaf nodes; this is the limit of the desired depth for each tree [16]. If the maximum depth of the decision tree continues to increase, then the model's performance during the training set increases continuously. However, at a certain point, the accuracy began to decline rapidly. A criterion is a parameter that measures the quality of the split. min_sample_split is the requisite minimum number of observations required at each node in order to initiate a split. The variable min_sample_leaf denotes the minimum number of samples required in a leaf node following the splitting of a node. The variable $max_features$ represents the upper limit on the number of features that can be considered for each tree within a Random Forest (RF) model. Moreover, we can set optional hyperparameters like $splitter$, $min_weight_fraction_leaf$, and max_leaf_nodes [17].
2) **Support Vector Machine (SVM)** is a machine learning algorithm with a supervised learning approach. SVM works by finding the best separator or hyperplane function to partition data points. The optimal hyperplane is a boundary separating classes with the maximum margin. Several critical hyperparameters that need to be tuned contribute to the hyperplane on the n-dimensional, namely the parameter penalty C, kernel function, gamma, degree, and epsilon. The parameter C influences the balance between the smoothness of decision boundaries and correctly classifying training points. The kernel function selects the type of hyperplane used to split the data. The most popular kernel functions available in sci-kit-learn are linear, polynomial, radial basis function, and sigmoid [18]. Gamma determines how far the influence of one sample training dataset is. The degree parameter controls the decision boundary flexibility. Epsilon in the support vector regression (SVR) model is a hyperparameter that describes the error between the distance and the loss function [19].
3) **K-nearest neighbor (KNN)** is a simple machine learning technique utilized to address classification and regression tasks. This algorithm works on the principle that any data points close to each other will be in the same class. In other words, KNN classifies new data points based on similarity. The closest number of data selected or k is the most critical hyperparameter set by the user [20]. If k is too big, the model will be overfitting. However, if k is too small, the model will be underfitting. In addition, there are several optional hyperparameters in KNN: weight functions used in predictions, power parameters for Minkowski metrics or p, and algorithms used to calculate nearest neighbors.

III. Research Method

In this section, we described our research methodology, which is the practical application of the hyperparameter optimization technique with PSO for four machine learning classification algorithms on two different data sets. We conducted these experiments to put theory into practice in Section II by developing an experimental code based on the study [3].

A. Datasets

We use some UC Irvine machine learning repository datasets, i.e., iris [10] and heart failure clinical records data [11]. The iris dataset is one of the most widely used and well-known datasets in data science, statistics, and machine learning [21]. The Iris dataset has several unique characteristics that make it suitable for machine learning, such as a multivariable dataset with four numerical attributes measuring various aspects of the iris flower, usually used for classification tasks, and consisting of three classes with 50 samples for a total of 150 pieces in the dataset. This data consists of five features,

Authorized licensed use limited to: BMS College of Engineering. Downloaded on October 24,2026 at 07:14:19 UTC from IEEE Xplore. Restrictions apply.

with four as the results of measurements and one Iris class feature of three species as targets, as shown in Table I. Edgar Anderson, a botanist, collected this data from measurements of two Iris species, namely Iris Versicolor and Iris Setosa, in 1935. Later, Fisher added information on measurement data from Iris Virginica at different locations and times.

TABLE I
IRIS DATA

Feature	Description
$sepal_length$	length of Iris sepal (in cm)
$sepal_width$	width of Iris sepal (in cm)
$petal_length$	length of Iris petal (in cm)
$petal_width$	width of Iris petal (in cm)
$class$	three species of Iris (Iris Setosa, Iris Versicolor, Iris Virginica)

The heart failure clinical records data is the medical records of 299 heart failure patients at Allied Hospitals in Pakistan, consisting of 194 male and 105 female patients aged 40 to 95 years. The Faisalabad Institute of Cardiology collected the data from April to December 2015. The dataset consists of 13 features that describe the patient's body, clinical information, and lifestyle, as shown in Table II. The death event feature was the target of the classification problem, which stated if the patient died or survived before the mean end of the follow-up period of 130 days. This dataset is often used for data analysis, risk factor exploration, and machine learning to understand factors associated with clinical heart failure. The aim was to identify significant predictors of death from heart failure and build a prediction model.

TABLE II
HEART FAILURE CLINICAL RECORDS DATASET FEATURES

Feature	Description
age	the patients' age (in years)
sex	man or woman (binary)
$time$	follow-up time (in days)
$anemia$	the patients' anemia status (boolean)
$high_blood_pressure$	the patients' hypertension status (boolean)
$creatinine_phosphokinase$	the CPK enzyme level in the patients' blood (mcg/L)
(CPK)	
$diabetes$	the patients' diabetes status (boolean
$ejection_fraction$	the percentage of the patients' blood that leaves the heart at every beat (%)
$platelets$	the patients' blood's platelets level (kilo platelets/mL)
$serum_creatinine$	the serum creatinine levels in the patients' blood (mg/dL)
$serum_sodium$	the serum sodiums level in the patients' blood (mEq/L)
$smoking$	the patients' smoking status (boolean)
$death_event$	the patients' deceased status during follow-up time (boolean)

B. Experimental setup

To boost the performance of the machine learning model, we need to know the critical hyperparameters in each algorithm and how to set them effectively. The key hyperparameters can vary depending on the specific machine-learning algorithm used. Table III shows various hyperparameters, types, and search spaces for different machine learning classifiers used in this research.

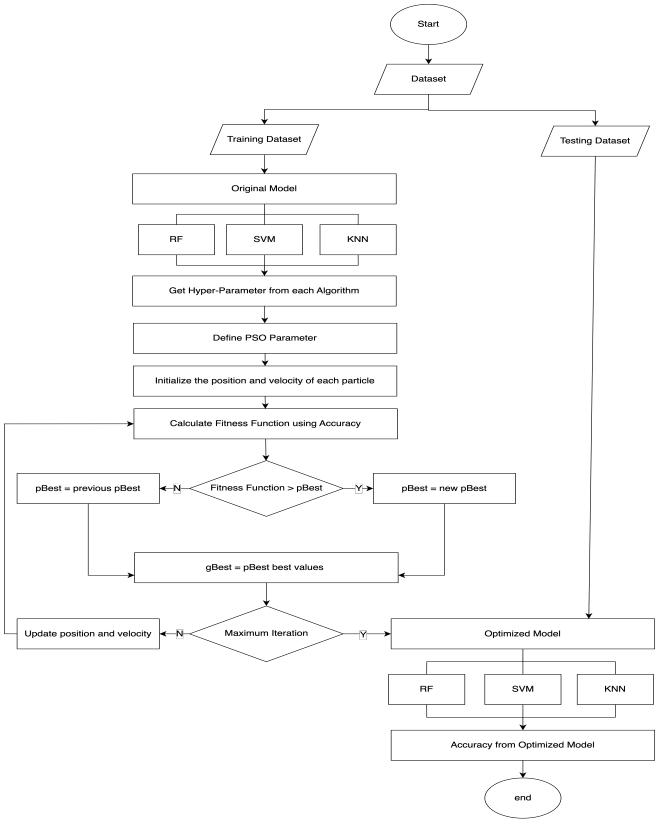


Fig. 1. Apply PSO to Machine Learning Algorithms

TABLE III
HYPERPARAMETERS OF TESTED CLASSIFIER ML MODELS

ML Model	Hyperparameter	Type	Search Space
RF	$n_estimators$	Discrete	[10,100]
	$max_features$	Discrete	[1,64]
	max_depth	Discrete	[5,50]
	$min_samples_split$	Discrete	[2,11]
	min_sample_leaf	Discrete	[1,11]
	$criterion$	Categorical	['gini', 'entropy']
SVM	C	Continuous	(0, 50)
	$kernel$	Categorical	['linear', 'poli' 'rbf', 'sigmoid']
KNN	$n_neighbors$	Discrete	[1, 20]

C. Apply PSO to Machine Learning Algorithms

Figure 1 illustrated the steps involved in fine-tuning the hyperparameters of the machine learning algorithm used in this study to achieve improved model performance. The steps are divided into several key sections, each representing a crucial phase of the hyperparameter optimization process in this study:

1) **Initialization**: The process begins with the initialization of the PSO algorithm. A population of particles is randomly initialized within the search space of hyperparam-

Authorized licensed use limited to: BMS College of Engineering. Downloaded on October 24,2026 at 07:14:19 UTC from IEEE Xplore. Restrictions apply.

eters. Each particle represents a potential hyperparameter configuration for the machine learning algorithm.

2) **Evaluation**: The machine learning model is trained and validated on the dataset using the respective hyperparameter, and the corresponding evaluation metric is calculated. We use classification performance metrics to understand the extent to which the model is accurate and effective in performing classification. The performance of each particle's hyperparameter configuration is evaluated using an accuracy metric with the equation below:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

The variables TP, TN, FP, and FN represent the quantities of correctly predicted positive instances, correctly predicted negative instances, incorrectly predicted positive instances, and incorrectly predicted negative instances, respectively.

3) **Local Best** ($pBest$): Next step is each particle compares its own performance (accuracy metric) with that of its previous best configuration. The best hyperparameter configuration for each particle is updated based on the individual best performance. In this study, the best hyperparameter configuration for each particle is updated if the current $pBest$ is greater than the previous $pBest$.
4) **Global Best** ($gBest$): The particle with the best hyperparameter configuration (highest accuracy) found so far is identified as the $gBest$. The $gBest$ particle represents the optimal hyperparameter values discovered up to the current iteration process.
5) **Updating Velocity**: Next, the particles update their velocities, which determine their movement in the hyperparameter space. This movement is based on their current position, the global best position ($gBest$), and their individual best position ($pBest$). The velocity updates ensure a balance between exploration and exploitation, enabling the particles to navigate search space effectively.
6) **Updating Position**: After updating velocities, the particles' positions in the hyperparameter space are adjusted accordingly. The updated positions represent new potential hyperparameter configurations for the machine learning model.
7) **Termination Criteria**: The optimization process continues iteratively until a maximum number of iterations is met.
8) **Final Result**: Upon termination, the $gBest$ particles' hyperparameter configuration represents the optimized set of hyperparameter for the machine learning model. This new configuration achieves the best performance based on the evaluation metric and is chosen as the final result of the hyperparameter optimization process.

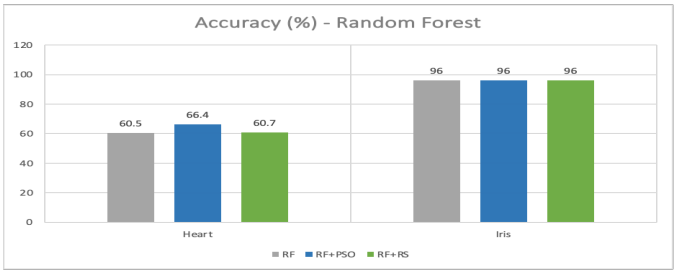


Fig. 2. Accuracy of the Random Forest model before and after adding the PSO and RS to the Heart failure clinical record and Iris datasets

IV. Results and Discussions

A. Optimization Result

In this result and discussion session, we present the outcomes of the hyperparameter optimization for each algorithm used and analyze its impact on the model's performance, as shown in Table IV. Accuracy is used as the measurement for determining the performance of the model, and for additional comparison, in addition to the results of optimization using PSO, optimization using Random Search (RS) is also shown here.

1) **Random Forest:** In Figure 2, we present the results of the hyperparameter optimization process for the Random Forest algorithm on the Heart Failure and Iris dataset. The hyperparameters considered for optimization include the number of trees ($n_estimators$), how many features to consider when determining the best split ($max_features$), the deepest point of each tree (max_depth), the least amount of samples necessary to split an internal node ($min_samples_split$), the smallest number of samples needed at each leaf node ($min_samples_leaf$), the function that measure a split's quality ($criterion$). After conducting the hyperparameter optimization process using both Particle Swarm Optimization (PSO) and Random Search for the Random Forest model, we observed notable improvements in accuracy. Particularly, on the Heart Failure Clinical Records Dataset, the accuracy increased by 5.9% after applying PSO, showcasing the effectiveness of this optimization technique in efficiently exploring the hyperparameter space and finding better configurations. Meanwhile, Random Search led to a more modest accuracy improvement of 0.2% on the same dataset. However, the original Random Forest model and the versions enhanced with PSO and Random Search yielded the same accuracy for the Iris dataset.
2) **Support Vector Machine:**The SVM hyperparameters considered for optimization are the regularization parameter (C) with a search space between 0 and 50 and the choice of the kernel function. For the Heart Failure Clinical Records Dataset, the accuracy after the hyperparameter optimization remained the same as before. On the other hand, for the Iris dataset, both Particle Swarm Optimization (PSO) and Random Search increased the

Authorized licensed use limited to: BMS College of Engineering. Downloaded on October 24,2026 at 07:14:19 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
OPTIMIZATION RESULT

Dataset	ML model	Accuracy (%)			Computation Time (ms)		
		Baseline	Baseline+PSO	Baseline+RS	Baseline	Baseline+PSO	Baseline+RS
Heart	RF	60.5	66.4	60.7	3059.10	9951.94	9038.72
	SVM	67.8	67.8	67.8	47.637	751.88	511.57
	KNN	62.2	67.8	67.8	51.9325	7064.458	5154.43
Iris	RF	96	96	96	592.082	9824.29	9768.02
	SVM	96	97.3	98	58.11	601.04	494.02
	KNN	98	98.7	98.7	26.39	6256.74	5888.29

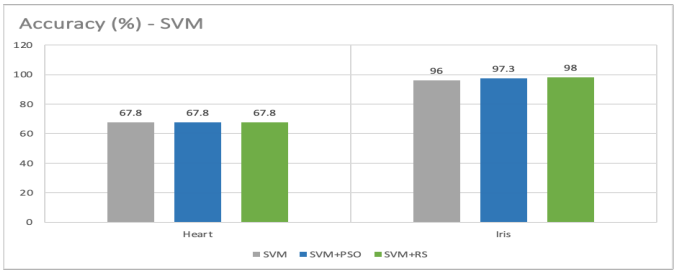


Fig. 3. Accuracy of the Support Vector Machine model before and after adding the PSO and RS method to the Heart failure clinical record and Iris datasets

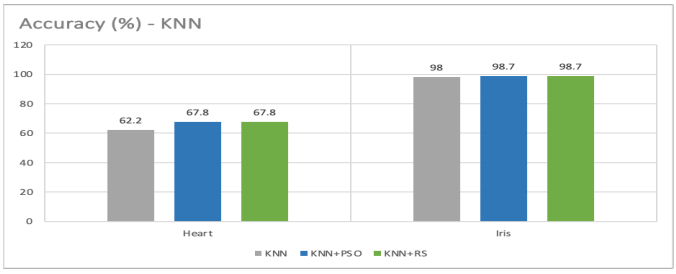


Fig. 4. Accuracy of the K-Nearest Neighbor model before and after adding the PSO method to the Heart failure clinical record and Iris datasets

accuracy of the SVM model. PSO improved accuracy by 1.3%, while Random Search achieved a higher improvement of 2%.

3) **K-Nearest Neighbor:** Figure 4 depicts the results of the hyperparameter optimization process for the K-Nearest Neighbors (KNN) algorithm on both the Heart Failure Clinical Records Dataset and the Iris dataset. The hyperparameter considered for optimization is the number of neighbors (K), which defines the number of data points to consider when making predictions. Despite the simplicity of the optimization process, both Particle Swarm Optimization (PSO) and Random Search successfully increased the accuracy of the KNN model on both datasets. For the Heart Failure Clinical Records Dataset, the accuracy improved by 5.6% after applying both PSO and Random Search. Similarly, on the Iris dataset, PSO and Random Search led to accuracy improvements of 0.7%.

B. Discussion

The results of the hyperparameter optimization process demonstrated the importance of careful parameter tuning in achieving improved model performance. Each algorithm exhibited varying degrees of sensitivity to hyperparameter settings, and the effectiveness of optimization techniques depended on the dataset's complexity.

For the Random Forest algorithm, Particle Swarm Optimization (PSO) was highly effective in efficiently exploring the hyperparameter space and finding configurations that significantly increased accuracy on the Heart Failure Clinical Records Dataset. Random Search also provided some improvement, though smaller. However, the initial hyperparameter settings for the Iris dataset already produced optimal results, making further optimization unnecessary.

In the case of SVM, PSO and Random Search did not lead to any improvement in accuracy on the Heart Failure Clinical Records Dataset. However, on the Iris dataset, both optimization techniques provided accuracy improvements, demonstrating their effectiveness in fine-tuning SVM for specific datasets.

For the KNN algorithm, both PSO and Random Search efficiently increased accuracy on both datasets. This highlights the importance of selecting the appropriate number of neighbors (K) for achieving better performance in KNN, particularly for complex datasets like the Heart Failure Clinical Records Dataset.

To evaluate the effectiveness of PSO and Random Search as hyperparameter optimization techniques, we recognize the significance of statistical tests to assess the robustness of our findings. We conducted statistical tests to determine the p-values associated with the improvements achieved by PSO and RS in comparison to the baseline model. For the Heart Failure Clinical Records Dataset, the calculated p-value for PSO is 0.113, whereas for the Random Search method is 0.296. For the Iris dataset, the p-value for PSO on the Iris dataset is 0.457, while for RS, it is 0.451. These p-values are essential indicators of statistical significance. In this context, a p-value below a predetermined significance level would suggest that the observed improvements are statistically significant. In our case, the p-values for both method and dataset exceed the conventional significance level of 0.05, indicating that the observed improvements might not be statistically significant.

Figure 5 provide insights into the computational time required for various machine learning models when employing hyperparameter optimization techniques. For the Heart dataset,

Authorized licensed use limited to: BMS College of Engineering. Downloaded on October 24,2026 at 07:14:19 UTC from IEEE Xplore. Restrictions apply.

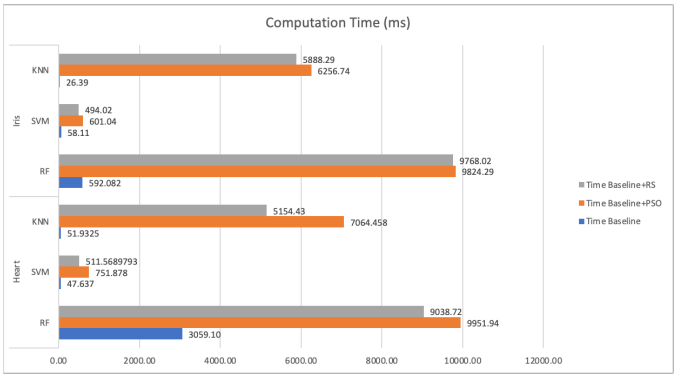


Fig. 5. Computation Time

we can observe that PSO and RS often demand significantly more computation time compared to the baseline models. This increase in computation time is a trade-off for the substantial improvements in model accuracy achieved with these optimization techniques. For instance, the RF model's accuracy improved from 60.5% to 66.4% with PSO but required more computation time (9951.94 ms) compared to the baseline (3059.10 ms).Conversely, on the Iris dataset, the computational time differences are less pronounced, and in some cases, optimization methods even reduce the computation time while enhancing accuracy. For instance, the SVM model's accuracy improved from 96% to 97.3% with PSO, and the computation time decreased from 58.11 ms (baseline) to 601.04 ms (PSO).

Overall, the results underscore the significance of dataset-specific hyperparameter tuning. The same optimization technique may yield varying outcomes on different datasets, emphasizing the importance of understanding the data characteristics and algorithm behavior to make informed hyperparameter choices.

V. Conclusions

This study aims to explore the significant role of hyperparameter tuning in optimizing machine learning algorithms. By employing Particle Swarm Optimization (PSO) and comparing it with Random Search, we have explored its effectiveness in fine-tuning three popular machine learning algorithms (Random Forest, Support Vector Machine (SVM), and K-Nearest Neighbors (KNN)) across two diverse datasets (the Heart Failure Clinical Records Dataset and the Iris dataset). While our findings indicate that PSO and Random Search have varying impacts on different algorithms and datasets, they underscore the critical importance of selecting the right optimization technique for specific scenarios. Our study showcases that PSO has the potential to efficiently navigate the hyperparameter space and yield substantial improvements in model accuracy, as evidenced in the case of the Heart Failure Clinical Records Dataset. However, the statistical significance test indicates that the improvement might not be statistically significant.

Looking ahead, this study opens up promising development for future research. Researchers and practitioners can delve deeper into exploring and comparing different hyperparameter optimization techniques, considering the unique characteristics of the dataset and the algorithms to make informed decisions that maximize model performance. The ultimate goal is to create accurate and reliable machine-learning models that can effectively address complex real-world challenges across various domains.

References

[1] R. Boutaba, M. A. Salahuddin, N. Limam, S. Ayoubi, N. Shahriar, F. Estrada-Solano, and O. M. Caicedo, "A comprehensive survey on machine learning for networking: evolution, applications and research opportunities," *Journal of Internet Services and Applications*, vol. 9, no. 1, pp. 1–99, 2018.

[2] I. H. Sarker, "Machine learning: Algorithms, real-world applications and research directions," *SN computer science*, vol. 2, no. 3, p. 160, 2021.

[3] L. Yang and A. Shami, "On hyperparameter optimization of machine learning algorithms: Theory and practice," *Neurocomputing*, vol. 415, pp. 295–316, 2020.

[4] Y. Shi and R. C. Eberhart, "Parameter selection in particle swarm optimization," in *Evolutionary Programming VII: 7th International Conference, EP98 San Diego, California, USA, March 25–27, 1998 Proceedings 7*. Springer, 1998, pp. 591–600.

[5] D. Boeringer and D. Werner, "Particle swarm optimization versus genetic algorithms for phased array synthesis," *IEEE Transactions on Antennas and Propagation*, vol. 52, no. 3, pp. 771–779, 2004.

[6] Y. Shi and R. Eberhart, "Empirical study of particle swarm optimization," in *Proceedings of the 1999 Congress on Evolutionary Computation-CEC99 (Cat. No. 99TH8406)*, vol. 3, 1999, pp. 1945–1950 Vol. 3.

[7] L. Breiman, "Random forests," *Machine learning*, vol. 45, pp. 5–32, 2001.

[8] S. Suthaharan and S. Suthaharan, "Support vector machine," *Machine learning models and algorithms for big data classification: thinking with examples for effective learning*, pp. 207–235, 2016.

[9] G. Guo, H. Wang, D. Bell, Y. Bi, and K. Greer, "Knn model-based approach in classification," in *On The Move to Meaningful Internet Systems 2003: CoopIS, DOA, and ODBASE: OTM Confederated International Conferences, CoopIS, DOA, and ODBASE 2003, Catania, Sicily, Italy, November 3-7, 2003. Proceedings*. Springer, 2003, pp. 986–996.

[10] R. A. Fisher, "Iris," UCI Machine Learning Repository, 1988, DOI: https://doi.org/10.24432/C56C76.

[11] "Heart failure clinical records," UCI Machine Learning Repository, 2020, DOI: https://doi.org/10.24432/C5Z89R.

[12] J. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization," *Journal of machine learning research*, vol. 13, no. 2, pp. 281–305, 2012.

[13] Q. Yao, M. Wang, Y. Chen, W. Dai, Y.-F. Li, W.-W. Tu, Q. Yang, and Y. Yu, "Taking human out of learning applications: A survey on automated machine learning," *arXiv preprint arXiv:1810.13306*, 2018.

[14] D. Wang, D. Tan, and L. Liu, "Particle swarm optimization algorithm: an overview," *Soft computing*, vol. 22, pp. 387–408, 2018.

[15] S. S. Priyadharsini and S. E. Rajan, "An efficient method for the removal of ecg artifact from measured eeg signal using pso algorithm," *Int J Adv Soft Comput Appl*, vol. 6, no. 1, pp. 1–19, 2014.

[16] L. Yang, A. Moubayed, I. Hamieh, and A. Shami, "Tree-based intelligent intrusion detection system in internet of vehicles," in *2019 IEEE global communications conference (GLOBECOM)*. IEEE, 2019, pp. 1–6.

[17] S. Sanders and C. Giraud-Carrier, "Informing the use of hyperparameter optimization through metalearning," in *2017 IEEE International Conference on Data Mining (ICDM)*. IEEE, 2017, pp. 1051–1056.

[18] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg *et al.*, "Scikit-learn: Machine learning in python," *the Journal of machine Learning research*, vol. 12, pp. 2825–2830, 2011.

[19] H. Drucker, C. J. Burges, L. Kaufman, A. Smola, and V. Vapnik, "Support vector regression machines," *Advances in neural information processing systems*, vol. 9, 1996.

[20] W. Zuo, D. Zhang, and K. Wang, "On kernel difference-weighted k-nearest neighbor classification," *Pattern Analysis and Applications*, vol. 11, pp. 247–257, 2008.

[21] A. Unwin and K. Kleinman, "The iris data set: In search of the source of virginica," *Significance*, vol. 18, 2021.

Authorized licensed use limited to: BMS College of Engineering. Downloaded on October 24,2026 at 07:14:19 UTC from IEEE Xplore. Restrictions apply.